

CN451 Network Programming Final Project Guide

This project automates a simulated network, monitors router CPU, sniffs packets, and runs a TCP chat application across the lab hosts.

Topology

The topology follows topology.md:

Role	IP address	Notes
VyOS router	192.168.1.254	Routes LAN 1 and LAN 2
Arista switch	192.168.1.250	LAN 1 switch
MikroTik switch	192.168.2.250	LAN 2 switch
Chat server host	192.168.2.100	Run Chat-System/server.py here
Chat client 1	192.168.2.10	Connects to 192.168.2.100:5050
Chat client 2	192.168.1.11	Connects to 192.168.2.100:5050

Role	IP address	Notes
Chat client 3	192.168.1.10	Connects to 192.168.2.100:5050

Files

File	Purpose
config/devices.txt	Device inventory: name, IP address, and Netmiko device type
config/login.txt	Login data for each network device
config/commands.txt	Commands to apply to each device
src/validator.py	Validates the three input text files
src/network.py	ICMP reachability and SSH/Telnet Netmiko connection logic
main.py	Task 2 automation runner with Rich status output

File	Purpose
src/cpu_monitor.py	Task 3 CPU polling, logging, and graph generation
src/packet_sniffer.py	Task 4 Scapy packet sniffer
src/chat/server.py	Task 6 TCP chat server
src/chat/client.py	Task 6 TCP chat client

Setup

Create or activate a Python environment, then install dependencies:

```
pip install -r requirements.txt
```

For packet sniffing, run the sniffer as root or with the needed capture permissions.

Task 1: Simulated Network

Build the topology in GNS3 or the simulator of your choice using:

- 1 VyOS router
- 1 Arista switch
- 1 MikroTik switch
- 3 Alpine client hosts
- 1 Alpine server host

Configure the router interfaces as shown in topology.md:

- eth1: 192.168.1.254/24
- eth2: 192.168.2.254/24

The server should use 192.168.2.100/24. The clients should use:

- 192.168.2.10/24
- 192.168.1.11/24
- 192.168.1.10/24

Each host should have the correct default gateway for its subnet.

Task 2: Network Automation

Run:

```
python main.py
```

What it does:

1. Reads config/devices.txt, config/login.txt, and config/commands.txt.
2. Validates file structure and IPv4 addresses.
3. Pings each device before attempting automation.
4. Connects with SSH first.
5. If SSH times out or fails during handshake, tries Telnet when Netmiko has a matching Telnet driver.
6. Applies each device's commands.
7. Shows a Rich summary table with reachability, transport, and result.

Handled failures:

- Missing input files
- Malformed inventory, login, or command lines
- Invalid IP addresses
- ICMP unreachable devices
- SSH/Telnet timeout or handshake failure
- Authentication failure
- Socket and protocol failures

Task 3: CPU Monitoring

Run:

```
python -m src.cpu_monitor
```

What it does:

1. Loads the VyOS router from config/devices.txt.
2. Connects through SSH first, then Telnet if SSH fails.

3. Polls run show system uptime.
4. Extracts the 1-minute load average.
5. Writes real-time samples to data/cpu_log.txt.
6. Creates assets/cpu_progress.png.

If the router output does not contain a CPU/load value, the script warns and keeps polling. If no samples are collected, it does not create a misleading graph.

Task 4: Packet Sniffing

Run on a host where Scapy can capture traffic:

```
If inside of the vms
sudo python -m src.packet_sniffer -i eth0 -c 20
```

```
If from the host machine
sudo python -m src.packet_sniffer -i virbr0 -c 20
```

Optional filter example for the chat app:

```
sudo python -m src.packet_sniffer -i eth0 -c 20 -f "tcp port 5050"
```

The sniffer displays a live Rich table with time, source, destination, protocol, and packet length.

Task 5: Extra Features

The project includes these extra features:

- Rich terminal UI for clearer tables, panels, progress, and timestamps.
- File upload/download in the chat application.
- Private messages in the chat application.
- Rock-paper-scissors game between chat users.
- CPU graph generation with Matplotlib.

Task 6: Chat System

Use the src/chat/ directory.

Run the server on 192.168.2.100

On the server host:

```
python -m src.chat.server --host 0.0.0.0 --port 5050
```

The server binds to all interfaces by default, so clients can connect to 192.168.2.100:5050. If you need to copy the server file to the server host from another machine, use SSH/SCP with the server host's real username:

```
scp src/chat/server.py USERNAME@192.168.2.100:~/server.py
```

Then on 192.168.2.100:

```
python ~/server.py --host 0.0.0.0 --port 5050
```

Run each client

On each client host:

```
python -m src.chat.client
```

The client defaults to 192.168.2.100:5050. You can also pass the server explicitly:

```
python -m src.chat.client 192.168.2.100 5050
```

Chat commands:

Command	Description
/help	Show commands
/users	List online users
/pm <username> <message>	Send a private message

Command	Description
/rps <username>	Start rock-paper-scissors
rock, paper, scissors	Send an RPS move
/files	List uploaded server files
/upload <path>	Upload a file under 150 KB
/download <filename>	Download a file
/quit	Disconnect safely

Handled chat failures:

- Invalid username
- Duplicate username
- Broken JSON payloads
- Oversized protocol messages
- Oversized uploads
- Missing upload/download files
- Client disconnect during a game
- Server disconnect while a client is running
- Bad command syntax

Recommended Demo Flow

1. Show the topology and IP addresses.
2. Run `python main.py` and explain the input files.
3. Run `python -m src.cpu_monitor` and show `data/cpu_log.txt` plus `assets/cpu_progress.png`.

4. Run `sudo python packet_sniffer.py -i eth0 -c 20 -f "tcp port 5050"`.
5. Start the chat server on 192.168.2.100.
6. Start three clients from 192.168.2.10, 192.168.1.11, and 192.168.1.10.
7. Demonstrate public chat, private chat, /users, file upload/download, and RPS.

Troubleshooting Notes

If `sudo python packet_sniffer.py` says Scapy is not installed while `pip install scapy` says it is installed, `sudo` is probably using the system Python instead of the project venv Python. The sniffer now detects that and retries with `venv/bin/python` when available.

If Arista fails with Pattern not detected, the device prompt probably changed during automation or an interactive command did not return to the prompt. The command file now avoids the interactive EOS banner command, and the automation applies hostname commands last with a generic `#` prompt terminator.

If CPU monitoring says no CPU value was found, the VyOS command output did not match the old Linux-style load average pattern. The monitor now tries several VyOS commands and parses both load-average and idle-percentage formats.